

Guide d'optimisation des tokens

Réduire la consommation de tokens = réduire les coûts, améliorer la vitesse, et éviter les dépassements de contexte. Ce guide couvre toutes les techniques, du plus impactant au plus fin.

Comprendre ce qui consomme des tokens

Anatomie d'une requête

Requête totale = System Prompt + Historique de conversation + Nouveau message + Outils disponibles

▲

CLAUDE.md +
skills actives

▲

messages précédents
compressés ou non

▲

votre prompt
+ fichiers lus

▲

descriptions
des skills

Où vont les tokens (ordre d'impact)

Source	Impact typique	Levier d'action
Fichiers chargés dans le contexte	■■■■■ Très élevé	.claudeignore, Read ciblé
Historique de conversation long	■■■■ Élevé	/clear, sessions courtes
CLAUDE.md verbeux	■■■ Moyen	Concision, références externes
Skills chargées	■■ Faible-Moyen	Divulgateion progressive
Réponses de Claude	■ Variable	Prompt de contrainte

Technique 1 — .claudeignore : exclure le bruit à la source

Le `.claudeignore` empêche l'indexation automatique des fichiers non pertinents. C'est la technique la **plus impactante** pour les projets avec beaucoup de fichiers.

Économie typique : 40-80% des tokens d'indexation sur un projet Node.js

node_modules/
dist/
*.log
data/raw/

Estimation des économies :

Avant .claudeignore : 50 000 tokens de contexte initial
Après .claudeignore : 8 000 tokens de contexte initial
Économie : 84%

(chiffres illustratifs sur un projet Node.js moyen — varient fortement selon la taille du repo)

Voir le template complet : `templates/claudeignore/template.md`

Technique 2 — Prompt Caching : réutiliser le travail déjà fait

Le prompt caching évite de re-tokeniser les portions stables du contexte entre les requêtes.

Ce qui est cacheable

```
# Avec le SDK Python – marquer un bloc cacheable
{
  "role": "user",
  "content": [
    {
      "type": "text",
      "text": "Voici la documentation complète de l'API (5000 tokens)...",
      "cache_control": {"type": "ephemeral"} # ← marquage cache
    },
    {
      "type": "text",
      "text": "Question : comment paginer les résultats ?"
    }
  ]
}
```

Seuils de caching

Modèle	Tokens minimum pour cacher
Haiku 4.5	2 048 tokens
Sonnet 4.6	1 024 tokens
Opus 4.7/4.8	1 024 tokens

⚠ Ces seuils sont indicatifs et peuvent évoluer. Consulter la [documentation officielle Anthropic](#) pour les valeurs exactes sur les modèles utilisés.

Stratégie optimale

1. **CLAUDE.md** : stable → éligible au cache si le préfixe du system prompt est identique entre les requêtes (non garanti dans tous les contextes d'exécution)
2. **Documents de référence** : charger en début de session → éligibles au cache dès le 2^e appel
3. **Historique de conversation** : augmente à chaque tour → compresser régulièrement

Économie typique avec caching :

⚠ Le caching réduit le **coût de facturation**, pas le nombre de tokens traités. Le modèle traite toujours le même volume — les tokens cachés sont simplement facturés à ~10% du prix normal (*source : grille tarifaire Anthropic — cache read input tokens*).

Sans cache : 10 requêtes × 2 000 tokens system × tarif plein = 20 000 unités de coût
Avec cache : 2 000 (1^{re} requête, tarif plein)
+ 9 × 200 (cache hits, ~10% du tarif) = 3 800 unités de coût
Économie sur la facturation : ~81%

(chiffres illustratifs — varient selon le modèle et le tarif en vigueur)

Technique 3 — Divulgaration progressive dans les skills

Ne chargez que ce dont Claude a besoin au moment T.

Niveau 1 (system prompt permanent)
└ name + description = ~50 tokens/skill ← approximation

Niveau 2 (déclenché si skill activée)
└ corps du SKILL.md = 200-500 tokens ← ordre de grandeur

Niveau 3 (seulement si demandé explicitement)
└ references/*.md = 500-2000 tokens ← ordre de grandeur

À ne pas faire :

```
# SKILL.md avec 3000 tokens de contenu → chargé entier à chaque activation
## Règles JavaScript
[500 lignes de règles détaillées...]
## Règles Python
[500 lignes...]
## Règles Go
[500 lignes...]
```

À faire :

```
# SKILL.md = 200 tokens + renvoi vers references/
## Règles
Pour les règles détaillées par langage : voir references/checklists-par-langage.md
(charge uniquement la section du langage concerné)
```

Technique 4 — Lectures ciblées avec Read

Charger un fichier entier quand seules quelques lignes sont nécessaires est l'erreur la plus courante.

```
# Coûteux : lecture entière (fichier de 500 lignes = ~5000 tokens)
Read("src/auth/middleware.ts")

# Optimal : lecture ciblée (20 lignes = ~200 tokens)
Read("src/auth/middleware.ts", offset=45, limit=20)
```

Stratégie en 2 temps :

1. `Grep` pour localiser la ligne exacte
2. `Read` avec `offset` et `limit` pour lire uniquement ce contexte

```
Grep "validateToken" src/ → trouvé à src/auth/middleware.ts:52  
Read("src/auth/middleware.ts", offset=48, limit=30) → 30 lignes = ~300 tokens
```

Technique 5 — Compresser l'historique de conversation

Chaque échange s'accumule dans le contexte. Utilisez `/clear` pour repartir propre.

Quand faire `/clear`

Session > 10 échanges	→ envisager <code>/clear</code>
Contexte à > 50% de capacité	→ <code>/clear</code> recommandé
Changement de sujet majeur	→ <code>/clear</code> obligatoire
Début de revue d'un nouveau fichier	→ <code>/clear</code> pour repartir propre

Pattern "résumé avant clear"

Avant de faire `/clear`, demandez un résumé des décisions :

```
Vous : "Résume en 5 points les décisions architecturales prises dans  
       cette session. Je vais faire /clear ensuite."  
Claude : [résumé]  
Vous : /clear  
Vous : [relancer avec le résumé + le nouveau problème]
```

Technique 6 — **CLAUDE.md** concis

Chaque ligne de `CLAUDE.md` est chargée à chaque requête.

Règle des 150 lignes max :

< 50 lignes	→ excellent
50-150 lignes	→ acceptable
> 150 lignes	→ refactoriser vers des fichiers de référence

Refactorisation type :

```
# Avant (dans CLAUDE.md) – 80 lignes de règles ESLint
[règle 1]
[règle 2]
...
[règle 40]

# Après (dans CLAUDE.md) – 2 lignes
## Style
Voir `docs/code-style.md` pour les règles complètes.
Utiliser `npm run lint` pour vérifier.
```

Technique 7 — Contraindre la longueur des réponses

Si vous n’avez besoin que d’une réponse courte, le dire économise des tokens de sortie.

```
# Prompt verbeux → réponse de 500 tokens
"Peux-tu analyser cette fonction ?"

# Prompt contraint → réponse de 50 tokens
"Quel est le problème dans cette fonction ? Réponds en une phrase."

# Prompt structuré → réponse formatée et dense
"Identifie les 3 problèmes principaux. Format : `- fichier:ligne – problème`"
```

Technique 8 — Choisir le bon modèle pour la taille de contexte

Scénario	Modèle optimal
Tâche courte, fichier < 100 lignes	Haiku 4.5
Tâche standard, fichier 100–500 lignes	Sonnet 4.6
Analyse de base de code, > 10 fichiers	Sonnet 4.6 ou Opus
Refactoring complet d’un module	Opus 4.7/4.8

Le coût par token de Haiku est ~20× inférieur à Opus (*approximation — vérifier la [page pricing Anthropic](#) pour les valeurs exactes*). Pour les tâches répétitives et simples, Haiku est presque toujours le bon choix.

Tableau de bord : estimer la consommation avant d’agir

```
# Estimer les tokens d'un fichier (règle empirique : 1 token ≈ 4 caractères en anglais/code)
# Variable selon la langue (le français tokenise ~10-20% moins bien) – mesurer précisément : anthropic.com/tokenizer
wc -c src/auth/middleware.ts | awk '{print int($1/4), "tokens estimés"}'

# Estimer la taille totale du projet (ce que Claude pourrait charger)
find . -name "*.ts" -o -name "*.py" | xargs wc -c 2>/dev/null | \
  tail -1 | awk '{print int($1/4), "tokens max si tout est chargé"}'

# Lister les 10 plus gros fichiers (candidats .claudeignore)
find . -name "*.ts" -o -name "*.py" | \
  xargs wc -c 2>/dev/null | sort -rn | head -10
```

Anti-patterns courants

- ❌ Charger node_modules/ dans le contexte
→ 50M+ tokens *(ordre de grandeur – un node_modules typique contient 100-500 Mo de JS minifié)*. Toujours dans .claudeignore.
- ❌ Lire config.json entier pour un seul paramètre
→ Grep d'abord, Read ciblé ensuite.
- ❌ Garder une session ouverte toute la journée
→ Le contexte s'accumule. /clear entre les tâches distinctes.
- ❌ CLAUDE.md avec 400 lignes de specs techniques
→ Déplacer dans references/, garder 3 lignes dans CLAUDE.md.
- ❌ Demander une analyse complète d'un repo de 200 fichiers
→ Limiter à la zone concernée, utiliser Glob pour cibler.
- ❌ Répéter le contexte à chaque message
→ Le mettre en CLAUDE.md une fois pour toutes.

Récapitulatif des économies par technique

Technique	Économie typique	Effort
.claudeignore bien configuré	50–80% du contexte initial	Faible
Prompt caching	70–90% sur le coût des tokens stables	Moyen (SDK)
Lectures ciblées (Read + offset)	80–95% par lecture	Faible
Divulgaration progressive skills	60–80% des tokens skills	Faible
CLAUDE.md concis	20–40% du system prompt	Faible
Historique compressé (/clear)	30–60% par session longue	Faible
Modèle adapté (Haiku vs Opus)	80–95% du coût	Faible

Toutes les économies sont des approximations — les valeurs réelles dépendent du projet, du modèle et des tarifs en vigueur.

Références

- **Prompt caching** (seuils, tarifs cache) — docs.anthropic.com/fr/docs/build-with-claude/prompt-caching
- **Pricing Anthropic** (coûts par modèle, input/output/cache) — anthropic.com/pricing
- **Tokenizer** (mesurer précisément un texte ou un fichier) — anthropic.com/tokenizer